WIMBL: Crusoe + Nemotron Setup Spec

Instructions for the agent configuring Crusoe Managed Inference. Verified against Crusoe's serverless catalog on Oct 17, 2026.

Goal: run every WIMBL model call on an NVIDIA Nemotron model served by Crusoe Managed Inference (serverless). NVIDIA makes the model; Crusoe hosts it. Pick model IDs only from Crusoe's live catalog.

1. Endpoint

- **Mode:** Crusoe Managed Inference, serverless. Do not use Self-Serve Deployments (reserved capacity, not needed for a one-day build).
- **Base URL:** `https://api.inference.crusoecloud.com/v1/` (OpenAI-compatible chat completions).
- **Auth:** API key from the environment variable `CRUSOE_API_KEY`. Never hard-code or log it. Add it to `.env.example` as a blank placeholder only.
- **Model:** pass the exact catalog ID as the `model` parameter. The endpoint may deviate from the OpenAI spec in places, so test each feature you rely on.

2. Model routing by role

WIMBL role	Model ID (exact)	Why
Trip Controller, Independent Verifier, Learning Engine (the "parent")	nvidia/Nemotron-3-Super-120B-A12B	Largest Nemotron on Crusoe; 120B MoE, ~12B active. Most reasoning for freezing criteria, catching defects, promote/reject decisions.
Trip children: tickets, flights, hotels, transit, price watch	nvidia/Nemotron-3-Nano-30B-A3B or nvidia/nemotron-3.5-lightning-30b-a3b	Cheap and fast for parallel agents. Lightning is NVIDIA's newer model built for specialized always-on agent tasks. Benchmark both on one scenario; keep the winner.
Optional: image checks (ticket screenshots, venue maps)	nvidia/Nemotron-3-Nano-Omni-Reasoning-30B-A3B	Multimodal. Only add if time remains.

Keep routing in one config file (e.g. `backend/integrations/models.py`) so a model can be swapped without touching the controller or learning engine.

3. Catalog facts (Crusoe docs)

Model ID	Context on Crusoe	Price (third-party source)
nvidia/Nemotron-3-Super-120B-A12B	262k (NVIDIA markets 1M; plan for 262k)	~$0.30 / 1M input, ~$2.40 / 1M output (ComputePrices; confirm on Crusoe pricing page)
nvidia/Nemotron-3-Nano-30B-A3B	262k	Check Crusoe pricing page
nvidia/nemotron-3.5-lightning-30b-a3b	1M	Check Crusoe pricing page
nvidia/Nemotron-3-Nano-Omni-Reasoning-30B-A3B	262k	Check Crusoe pricing page

Crusoe's docs do not state whether these models support tool calling. Treat it as unknown until the smoke test passes.

4. Smoke tests (run first, before building)

Run against Super and against the chosen child model. Record pass/fail for each.

- **T1 Basic chat:** one short prompt returns text.
- **T2 Tool calling:** request with a `tools` array returns a `tool_calls` entry with valid JSON arguments.
- **T3 Structured output:** ask for JSON matching a small schema; parse it. If `response_format` is unsupported, fall back to prompt-instructed JSON plus validation and one repair retry.
- **T4 Concurrency:** fire 5 child requests in parallel; note any 429s and latency.

```
import os, json
from openai import OpenAI

client = OpenAI(base_url="https://api.inference.crusoecloud.com/v1/",
                api_key=os.environ["CRUSOE_API_KEY"])

tools = [{"type": "function", "function": {
    "name": "search_tickets",
    "description": "Find official ticket listings for a match",
    "parameters": {"type": "object",
        "properties": {"team": {"type": "string"}, "date": {"type": "string"}},
        "required": ["team"]}}}]

for model in ["nvidia/Nemotron-3-Super-120B-A12B",
              "nvidia/Nemotron-3-Nano-30B-A3B"]:
    r = client.chat.completions.create(
        model=model,
        messages=[{"role": "user",
                   "content": "Find tickets for FC Barcelona's next home match."}],
        tools=tools)
    msg = r.choices[0].message
    print(model, "tool_calls:", msg.tool_calls, "| model returned:", r.model)
```

If tool calling fails: use a JSON "action" protocol in the prompt (the model returns `{"tool": ..., "args": ...}`), parse it in code, and validate before executing.

5. Reliability

- **Retries:** on 429 or 5xx, retry with exponential backoff plus jitter (e.g. 1s, 2s, 4s, max 4 attempts). Do not retry 4xx auth or validation errors.
- **Concurrency cap:** limit parallel child calls with a semaphore (start at 4) to stay under rate limits.
- **Timeouts:** set a per-call timeout and count it against the run's tool-call budget.
- **Budgets:** controller enforces token, wall-clock, retry and tool-call budgets per run. On exhaustion, return a truthful partial result.

6. Run record (log on every model call)

```
run_id, agent_role, model_id_requested, model_id_returned, version_tag,
prompt_version, input_tokens, output_tokens, latency_ms, cost_estimate_usd,
tool_calls, status, error, timestamp
```

Use the `model` field returned in the response as the source of truth. This keeps N vs N+1 comparisons interpretable and supports the brief's "cost per verified success" metric.

7. Safety gate (in code, not a model)

- No purchase, booking, or payment call executes without explicit traveler approval. Implement as a hard check in the controller.

- NeMo Guardrails and NeMo Switchyard are out of scope for this build. Route by role via the config above.
- Optional, only if time allows: one Nemotron safety-model screen of the user request, and only if Crusoe serves one. Do not add a second provider for it.

8. Honesty rules for the demo

- Say: "WIMBL runs NVIDIA Nemotron [exact ID], served by Crusoe Managed Inference." Do not call Nemotron a Crusoe model.
- Only show model IDs, costs and metrics actually recorded in run logs. Never fabricate improvement numbers.
- Do not claim Switchyard, Guardrails or a 1M context on Super unless actually used and verified.
- Unverified items to confirm on the hackathon page: credit amount, prize categories, and any banned project categories.

9. Definition of done for this setup

- Smoke tests T1 to T4 recorded for Super and the chosen child model.
- Model routing lives in one config file; IDs match Crusoe's catalog exactly.
- Retry/backoff and concurrency cap in place.
- Every call writes a run record with the returned model ID.
- Approval gate blocks any booking or payment action.

Sources

- Crusoe Serverless Inference docs: docs.crusoecloud.com/serverless-inference/index.html
- Strands Agents, Crusoe provider (base URL): strandsagents.com/docs/integrations/model-providers/crusoe/
- ComputePrices, Nemotron 3 Super on Crusoe: computeprices.com/providers/crusoe/models/nemotron-3-super-120b-a12b
- NVIDIA Nemotron: developer.nvidia.com/topics/ai/nemotron
- WIMBL Build Brief